Alignement de séquence

L'*alignement de séquences* est un problème algorithmique classique de bio-informatique visant typiquement à comparer deux séquences d'ADN dans l'idée que si deux séquences d'ADN représentant des gènes sont similaires, alors la fonction du gène est probablement la même.

Fdardel, CC BY-SA 3.0, via Wikimedia Commons

Décontextualisation : On peut représenter deux séquences comme deux chaînes de caractères T_1 et T_2 et l'objectif est d'étudier leur degré de similarité (ou dissimilarité). Il y a deux approches possibles :

- soit on étudie leur dissimilarité : on cherche donc à *minimiser* la *distance* $d(T_1, T_2)$ entre les deux chaînes ;
- soit on étudie leur similarité : on cherche donc à *maximiser* un score de ressemblance $s(T_1, T_2)$ entre les deux chaînes.

Dans cet exercice, on choisira la première option : *minimiser* la distance entre deux chaînes.

Distance d'édition et alignement

Pour mesurer la distance entre deux chaînes, on choisit d'utiliser la **distance d'édition**, aussi appelée *distance de Levenshtein*.

L'objectif est de passer d'une chaîne à l'autre au moyen de 3 opérations :

- insertion (d'un caractère c dans T_1)
- suppression (d'un caractère c dans T_1)
- substitution (d'un caractère c par un autre caractère c' dans T_1)

Chacune de ces opérations a un coût et la *distance d'édition*, notée dE, minimise la somme de ces coûts pour passer d'une chaîne à l'autre. Autrement dit, la distance d'édition est égale au nombre minimal de caractères qu'il faut insérer, supprimer, ou substituer pour passer d'une chaîne à l'autre.

Dans la suite, on supposera que les coûts des trois opérations sont tous égaux à 1 !!

Exemple : T_1 = SUCCES et T_2 = ECHECS. On peut considérer l'alignement suivant :

```
S U C C E - S
- E C H E C S
```

Les tirets (-) sur le premier mot correspondent à une supression, et les tirets sur le deuxième mot correspondent à une insertion. Si deux caractères sont alignés : soit il s'agit d'une substitution s'ils sont différents, soit il s'agit d'une correspondance s'ils sont égaux (le coût est nul dans ce cas).

Autrement dit, avec cet alignement, pour passer de SUCCES à ECHECS, on a effectué les opérations suivantes :

- suppression de S : **-**UCCES (coût = 1)
- substitution de U par E : **E**CCES (coût = 1)
- correspondance de C : E**C**CES
- substitution de C par H : EC**H**ES (coût = 1)
- correspondance de E : ECH**E**S
- insertion de C : ECHE**C**S (coût = 1)
- correspondance de S : ECHEC**S**

On obtient un coût total égal à 4 (en fait il s'agit d'un alignement optimal, donc $\mathrm{dE}(\mathrm{SUCCES}, \mathrm{ECHECS}) = 4$, comme on le verra plus tard).

Question 1 : T_1 = SUCCES et T_2 = ECHECS. On considère l'alignement suivant

```
S - U C C E - S
- E - C H E C S
```

1. Ecrivez les opérations pour passer de SUCCES à ECHECS avec cet alignement.
2. Quelle est le coût de cet alignement ? Cet alignement est-il meilleur que le précédent ?

Moralité : Plusieurs alignements sont possibles et parmis ces alignements, certains ont un coût minimal : il s'agit d'alignements optimaux dont le coût est égal à la distance d'édition entre les deux chaîne. Ainsi, trouver un alignement optimal est équivalent à déterminer la distance d'édition.

Relation de récurrence

Le calcul de la distance d'édition entre deux chaînes peut se faire à partir de celles de chaînes plus petites. Cela permet de dégager une relation de récurrence qui sera exploitée par les algorithmes calculant la distance d'édition.

Notations : Dans la suite, on notera:

- n_1 la longueur de T_1 ; n_2 la longueur de T_2
- $T_1[i]$ le i-ème caractère de T_1 ; $T_2[j]$ le j-ème caractère de T_2 (avec $1 \leq i \leq n_1$ et $1 \leq j \leq n_2$)
- $T_1[1..i]$ les i premiers caractères de T_1 ; $T_2[1..j]$ les j premiers caractères de la chaîne T_2, (avec $1 \leq i \leq n_1$ et $1 \leq j \leq n_2$).

Comment peut-on calculer $\mathrm{dE}(T_1[1..i], T_2[1..j])$ si on connait $\mathrm{dE}()$ pour des "bouts de textes" plus courts ?

Supposons avoir aligné $T_1[1..i]$ et $T_2[1..j]$ (les i premiers caractères de T_1 et les j premiers caractères de T_2) et cherchons le coût de cet alignement. On regarde l'alignement le plus à droite. Quatre cas se présentent :

- Suppression : $T_1[i]$ s'aligne sur « - ». Cela demande à avoir aligné $T_1[1..i-1]$ et $T_2[1..j]$ et supprimé $T_1[i]$. On a donc dans ce cas :
$$\mathrm{cout}(T_1[1..i], T_2[1..j]) = \mathrm{cout}(T_1[1..i-1], T_2[1..j]) + \underbrace{1}_{\text{coût suppression}}$$
- Insertion : « - » s'aligne sur $T_2[j]$. Cela demande à avoir aligné $T_1[1..i]$ et $T_2[1..j-1]$ et inséré $T_2[j]$. On a donc dans ce cas :
$$\mathrm{cout}(T_1[1..i], T_2[1..j]) = \mathrm{cout}(T_1[1..i], T_2[1..j-1]) + \underbrace{1}_{\text{coût insertion}}$$
- Substitution : $T_1[i]$ s'aligne sur $T_2[j]$ avec $T_1[i] \neq T_2[j]$. Cela demande d'avoir aligné $T_1[1..i-1]$ et $T_2[1..j-1]$ et substitué $T_1[i]$ par $T_2[j]$. On a donc dans ce cas :
$$\mathrm{cout}(T_1[1..i], T_2[1..j]) = \mathrm{cout}(T_1[1..i-1], T_2[1..j-1]) + \underbrace{1}_{\text{coût substitution}}$$
- Correspondance : $T_1[i]$ s'aligne sur $T_2[j]$ avec $T_1[i] = T_2[j]$. Cela demande d'avoir aligné $T_1[1..i-1]$ et $T_2[1..j-1]$ et c'est tout (on substitue le caractère par lui-même). On a donc dans ce cas :
$$\mathrm{cout}(T_1[1..i], T_2[1..j]) = \mathrm{cout}(T_1[1..i-1], T_2[1..j-1]) + \underbrace{0}_{\text{sans surcoût}}$$

Comme la distance d'édition cherche à minimiser le coût d'un alignement, il faut choisir la valeur minimale de coût à chaque fois (parmi les 4 possibilités). On en déduit la relation de récurrence suivante (on a regroupé les deux derniers cas ensemble en introduisant un coût de substitution qui vaut 1 ou 0 selon les cas) :

$$\mathrm{dE}(T_1[1..i], T_2[1..j]) = \min \begin{cases} \mathrm{dE}(T_1[1..i-1], T_2[1..j]) + 1 \\ \mathrm{dE}(T_1[1..i], T_2[1..j-1]) + 1 \\ \mathrm{dE}(T_1[1..i-1], T_2[1..j-1]) + \mathrm{sub}(T_1[i], T_2[j]) \end{cases}$$

où $\mathrm{sub}(a, b) = \begin{cases} 0 \text{ si } a = b \text{ (correspondance)} \\ 1 \text{ si } a \neq b \text{ (substitution)} \end{cases}$

Version récursive naïve

On peut utiliser cette relation de récurrence pour écrire l'algorithme récursif naïf correspondant. Voici une version avec des *slices* qui est plus rapide à écrire (mais dont le coût en espace est plus important).

```
# Version récursive naïve
def dE(t1, t2):
    if len(t1) == 0:
        return len(t2)
    elif len(t2) == 0:
        return len(t1)
    else:
        if t1[-1] == t2[-1]:
            sub = 0
        else:
            sub = 1
        return min(dE(t1[0:-1], t2) + 1, dE(t1, t2[0:-1]) + 1, dE(t1[0:-1], t2[0:-1]) + sub)
```

On peut constater que cela marche pour des mots assez courts

```
%time dE('SUCCES', 'ECHECS')
```

Que cela prend un peu plus de temps si on augmente un peu la longueur des mots.

```
%time dE('DESTRUCTION', 'CREATION')
```

Et que cela ne termine pas en un temps raisonnable (voire pas du tout) si les mots sont encore un peu plus longs (*Vous pourrez stopper l'exécution manuellement si besoin*).

```
%time dE('SOUSTRACTION', 'MULTIPLICATION')
```

Question 2 : Dessinez les 3 premiers niveaux de l'arbre d'appels récursifs si on lance `dE('SUCCES', 'ECHECS')`. *On pourra juste noter les deux chaînes en arguments pour gagner un peu de temps.*

> Vous devez constater qu'il y a des appels redondants, et qu'ils deviennent vite beaucoup trop nombreux dès que la longueur des chaînes augmente un peu. L'efficacité est catastrophique !

Programmation dynamique

Vous allez maintenant utiliser la programmation dynamique pour améliorer l'algorithme précédents. On ne traitera que la version itérative ascendante (la version récursive avec mémoïsation est laissée en exercice).

Tableau de mémorisation

Comme pour les exemples précédents, l'idée est d'utiliser un tableau pour stocker les réponses déjà connues pour ne pas les calculer plusieurs fois.

Dans le cas de l'alignement de séquence :

- on va utiliser un tableau `D` à deux dimensions de taille $(n1+1) \times (n2+1)$.
- Pour tous $1 \leq i \leq n_1$ et pour tous $1 \leq j \leq n_2$, on a : `D[i][j]` $= \mathrm{dE}(T_1[1..i], T_2[1..j])$ (l'élément en position (i, j) est la distance d'édition entre les i premiers caractères de T_1 et les j premiers caractères T_2)
- La solution $\mathrm{dE}(T_1, T_2)$ se trouve dans la case en bas à droite `D[n1][n2]`.

		0	1	2	3	4	5	6
		ε	E	C	H	E	C	S
0	ε							
1	S							
2	U							
3	C							
4	C							
5	E							
6	S							

← $\mathrm{dE}(T_1, T_2)$ est ici !

Remarques :

- On ajouté une première ligne et une première colonne qui permettront de simplifier le début du remplissage du tableau. Cette première ligne contient ε qui désigne un mot vide.
- Dans cet exemple, la case en position $(2, 4)$ c'est-à-dire `D[2][4]` contient la distance d'édition entre les deux premières caractères de `'SUCCES'` et les quatre premiers caractères de `'ECHECS'`, donc elle contient $\mathrm{dE}(\text{'SU'}, \text{'ECHE'})$.

Remplissage du tableau

- On peut remplir facilement la première ligne et la première colonne car on cherche à chaque fois la distance avec un mot vide : il faut insérer ou supprimer le nombre de lettres du second mot. On obtient alors :

		0	1	2	3	4	5	6
		ε	E	C	H	E	C	S
0	ε	0	1	2	3	4	5	6
1	S	1						
2	U	2						
3	C	3						
4	C	4						
5	E	5						
6	S	6						

- Il suffit de remplir ensuite le tableau ligne par ligne et de haut en bas en utilisant la relation de récurrence.

Question 3 : Exprimez `D[i][j]` en fonction de `D[i-1][j]`, `D[i][j-1]` et `D[i-1][j-1]` (avec `i` entre 1 et n_1, et `j` entre 1 et n_2) puis identifiez les 4 cases en question dans le tableau `D`.

Question 4 : Pour compléter une case du tableau on utilise soit celle à sa gauche, soit celle au-dessus, soit celle au-dessus à gauche en diagonale. Selon le cas, indiquez s'il s'agit d'une substitution, d'une suppression, d'une insertion ou d'une correspondance.

Question 5 : Vous devriez avoir tous les éléments pour compléter le tableau `D`. Faites-le et déduisez-en la distance d'édition entre les mots SUCCESS et ECHECS.

Question 6 : Utilisez un tableau pour déterminer la distance d'édition entre les mots NICHE et CHIENS.

Implémentation

Question 7 : Écrivez une fonction `dE(t1, t2)` qui renvoie la distance d'édition entre les chaînes `t1` et `t2` ainsi que le tableau `D` (permettra de vérifier!). Vous utiliserez la méthode itérative ascendante qui construit la table de programmation dynamique `D` comme évoqué dans les question précédentes. *On pourra créer le tableau `D` de départ avec des zéros puis initialiser la première ligne et la première colonne.*

Question 8 : Testez la fonction, en particulier pour vérifier vos réponses aux questions 5 et 6 (distances entre SUCCES et ECHECS, puis entre NICHE et CHIENS. Vous vérifierez également que l'on obtient des réponses rapides mêmes pour des mots plus longs.

Construction d'un alignement optimal

Pour trouver un alignement optimal entre T_1 et T_2 il suffit de *remonter* depuis `D[n1][n2]` vers `D[0][0]` (de la case en bas à droite à celle en haut à gauche) en considérant à chaque fois le "meilleur" des 3 voisins, c'est-à-dire celui par lequel on a pu arriver ! (il peut parfois y en avoir plusieurs). Il faut se rappeler (cf. question 4) que :

- Si on monte, il s'agit d'une suppression
- Si on va à gauche, il s'agit d'une insertion
- Si on va en diagonale, il s'agit d'une substitution ou d'une correspondance

Par exemple, le tableau `D` correspondant à l'alignement entre SUCCES et ECHECS est :

		0	1	2	3	4	5	6
		ε	E	C	H	E	C	S
0	ε	0	1	2	3	4	5	6
1	S	1	1	2	3	4	5	5
2	U	2	2	2	3	4	5	6
3	C	3	3	2	3	4	4	5
4	C	4	4	3	3	4	4	5
5	E	5	4	4	4	3	4	5
6	S	6	5	5	5	4	4	4

Suppression

Insertion

Substitution/Correspondance

← $\mathrm{dE}(T_1, T_2)$ est ici !

On peut alors construire un alignement optimal (en partant de la fin ou du début) :

```
S U C C E - S
- E C H E C S
```

Question 9 : Déterminez un autre alignement optimal de ce deux chaînes.

Question 10 : Utilisez le tableau de la question 6 pour trouver un alignement optimal entre NICHE et CHIENS. Y en a-t-il d'autres ?

Question 11 (exercice) : Écrivez une fonction `alignement_optimal(t1, t2)` qiu renvoie un alignement optimal entre les deux chaînes `t1` et `t2`. On peut bien sûr s'appuyer sur la fonction `dE` de la question 7.

Question 12 (exercice) : Écrivez une fonction permettant de calculer la distance d'édition entre deux chaînes en utilisant une version récursive avec mémoïsation.